Product Requirements Document

Image Captioning with Vision + Transformer

Project Name: Multimodal AI Image Captioning System
Version: 1.0
Date: February 12, 2026
Author: AI Development Team

---

1. Overview

Goal: Build an AI system that generates text descriptions from images.

Example:

- Input: Image of a dog in a park
- Output: "A brown dog playing with a ball in the park"

Success Metric: BLEU-4 score ≥ 0.25

---

2. System Architecture

```
IMAGE (224×224)
      ↓
┌─────────────────┐
│ VISION ENCODER  │   Extract visual features
│ (ResNet/ViT)    │
└─────────────────┘
      ↓
  Image Features
      ↓
┌─────────────────┐
│ TRANSFORMER     │   Generate text word by word
│ DECODER         │
└─────────────────┘
      ↓
"A dog in the park"
```

---

3. Project Flow

Step 1: Setup

- Install PyTorch, transformers, torchvision
- Download MS COCO or Flickr8k dataset
- Create project structure

Step 2: Data Pipeline

Image Processing:

```
Load image → Resize to 224×224 → Normalize → Tensor
```

Caption Processing:

```
"A dog plays" → Tokenize → ["<START>", "A", "dog", "plays", "<END>"]
              → Convert to IDs → [1, 45, 289, 567, 2]
              → Pad to max length
```

Output: DataLoader yielding batches of (images, captions)

Step 3: Build Models

Vision Encoder (choose one):

- **ResNet-50**: CNN-based, 2048×7×7 features, fastest
- **ViT-Base**: Transformer-based, 196×768 features, better accuracy

Transformer Decoder:

- 6 layers with self-attention and cross-attention
- 512-768 embedding dimension
- 8-12 attention heads
- Vocabulary size: 10,000-30,000 tokens

Integration:

```
class CaptionModel:
    encoder → image_features
    projection → match decoder dimension
    decoder(captions, image_features) → predictions
```

Step 4: Training

Training Loop:

```
For each batch:
  1. Encode images → visual_features
  2. Feed captions with teacher forcing:
     Input:  <START> A    dog   plays
     Target: A       dog  plays <END>
  3. Calculate cross-entropy loss
  4. Backpropagate and update weights
  5. Log loss and metrics
```

Key Parameters:

- Batch size: 32-64
- Learning rate: 1e-4
- Optimizer: Adam with warmup

- Epochs: 20-50

Step 5: Inference

Caption Generation:

```
1. Encode image → fixed visual features
2. Start with sequence = [<START>]
3. Loop:
   - Decoder predicts next word probabilities
   - Select word (greedy or beam search)
   - Append to sequence
   - Stop if <END> or max length
4. Return generated caption
```

Decoding Strategies:

- **Greedy**: Pick highest probability word (fast)
- **Beam Search**: Keep top-k candidates (better quality)

Step 6: Evaluation

Metrics:

- **BLEU-4**: N-gram overlap (target ≥ 0.25)
- **CIDEr**: Consensus scoring (target ≥ 1.0)
- **METEOR**: Semantic similarity

Analysis:

- Generate captions for test set
- Compare with reference captions
- Identify failure patterns
- Visualize attention weights

---

4. Requirements

Functional

- Load and preprocess image-caption datasets
- Extract visual features using pretrained encoder
- Generate captions autoregressively
- Support batch inference
- Compute evaluation metrics
- Save/load model checkpoints

Non-Functional

- Inference: < 1 second per image on GPU
- Training: Support batch size ≥ 32
- Memory: < 12GB GPU VRAM
- Handle corrupted images gracefully

---

5. Data Specification

Datasets:

- MS COCO: 118K training images, 5 captions each
- Flickr8k: 8K images (easier for testing)

Format:

- Images: JPG/PNG, resized to 224×224
- Captions: Plain text, 5-20 words typical
- Split: 80% train, 10% validation, 10% test

---

6. Model Configuration

Recommended Setup

Beginner:

- Encoder: ResNet-50 (frozen)
- Decoder: 6-layer transformer (512-dim)
- Total params: ~65M
- Expected BLEU-4: 0.22-0.25

Advanced:

- Encoder: ViT-B/16 (frozen)
- Decoder: GPT-2 Small (768-dim, 6 layers)
- Total params: ~150M
- Expected BLEU-4: 0.26-0.30

---

7. Technical Stack

Core:

- Python 3.8+
- PyTorch 2.0+ or TensorFlow 2.x
- transformers (HuggingFace)
- torchvision

Utilities:

- PIL (image loading)
- nltk (tokenization, metrics)
- numpy, pandas
- matplotlib (visualization)

Hardware:

- GPU: 8GB+ VRAM (16GB recommended)
- Storage: 50GB+
- RAM: 16GB+

---

8. Deliverables

Code Structure:

```
project/
├── data/
│   ├── dataset.py
│   └── preprocessing.py
├── models/
│   ├── encoder.py
│   ├── decoder.py
│   └── captioner.py
├── train.py
├── inference.py
├── evaluate.py
└── config.yaml
```

Outputs:

- Trained model checkpoint
- Evaluation report with metrics
- Sample generated captions
- Attention visualizations

---

9. Evaluation Criteria

Quantitative

Metric	Target	Meaning
BLEU-4	≥ 0.25	Word overlap with references
CIDEr	≥ 1.0	Consensus agreement
METEOR	≥ 0.25	Semantic similarity

Qualitative

- Captions are grammatically correct
- Main objects correctly identified
- Spatial relationships described accurately
- Diverse outputs (not repetitive)

---

10. Risks & Mitigation

Risk	Mitigation
Overfitting on small data	Use pretrained encoders, dropout, augmentation
Generic captions	Beam search, reinforcement learning fine-tuning
Out of memory errors	Reduce batch size, use gradient accumulation
Slow training	Mixed precision (FP16), freeze encoder

---

11. Future Extensions

- Multi-language captioning
- Controllable generation (style, length)
- Dense captioning (multiple regions)
- Video captioning
- Real-time inference optimization